

Playwright Automation — Chapitre 1

Introduction, installation, premiers tests, fixtures, configuration et assertions

Cette documentation est construite à partir des notes du cours. Elle reprend la progression et les exemples du chapitre, puis ajoute des explications pédagogiques pour faciliter l'apprentissage. Les compléments qui ne figurent pas explicitement dans les notes sont signalés comme tels.

1. Objectifs du chapitre

- Comprendre ce qu'est Playwright et son rôle dans l'automatisation des tests end-to-end.
- Créer un projet Playwright avec Node.js.
- Comprendre la structure de base d'un projet.
- Écrire un premier test avec `test()`.
- Comprendre `async/await` dans les tests Playwright.
- Comprendre la relation `browser` → `browser context` → `page`.
- Utiliser les fixtures `browser` et `page`.
- Comprendre le fichier `playwright.config.js`.
- Exécuter les tests en mode `headless` ou `headed`.
- Utiliser `test.only()` pour isoler un test.
- Introduire les assertions avec `expect()`.
- Comprendre les premières bases de l'exécution multi-navigateurs.

2. Qu'est-ce que Playwright ?

Playwright est un framework d'automatisation de navigateurs utilisé notamment pour tester des applications web. Dans ce chapitre, le cours se concentre sur l'automatisation et les tests end-to-end.

L'idée fondamentale est de piloter un navigateur comme le ferait un utilisateur : ouvrir une page, naviguer vers une URL, interagir avec l'interface et vérifier le résultat attendu.

Le cours renvoie également à la documentation officielle Playwright concernant l'actionability : playwright.dev/docs/actionability.

3. Préparer l'environnement

Le cours demande :

- Node.js
- Visual Studio Code (VS Code)
- Un projet Playwright vide

Initialisation indiquée dans le cours :

`npm init playwright`

L'assistant de création du projet permet notamment de choisir JavaScript, le dossier des tests, le workflow GitHub Actions et l'installation des navigateurs.

4. Structure du projet

Élément	Rôle
node_modules	Contient les dépendances installées du projet.
tests	Contient les tests end-to-end du projet.
package.json	Décrit le projet et ses dépendances. Le cours le compare au rôle de pom.xml dans un projet Maven.
playwright.config.js	Centralise la configuration Playwright utilisée pour exécuter les tests.

5. Premier test Playwright

Le cours présente `test()` comme la fonction permettant de déclarer un cas de test. Elle reçoit notamment un titre et une fonction de test.

```
import { test } from '@playwright/test';

test("First Playwright Test", async () => {
  // Playwright code
});
```

La structure générale est donc : `test("nom du test", async () => { ... })`. Le bloc contient les actions réalisées pendant le scénario.

6. Pourquoi async / await ?

Les opérations Playwright sont fréquemment asynchrones : navigation, création d'un contexte, création d'une page, actions utilisateur et assertions. Le cours insiste donc sur l'utilisation de `async` et `await` pour attendre la fin des opérations avant de poursuivre.

Exemple conceptuel :

1. Étape 1 : ouvrir le navigateur.
2. Étape 2 : saisir le nom d'utilisateur et le mot de passe.
3. Étape 3 : cliquer sur Submit.

`await` permet ici d'exprimer clairement : « attends que cette opération soit terminée avant de passer à la suivante ».

7. Browser, Browser Context et Page

C'est l'un des concepts essentiels du chapitre. Le cours montre deux façons de travailler : créer explicitement un contexte et une page, ou utiliser directement la fixture `page`.

7.1 Browser

`browser` représente le navigateur lancé pour l'exécution du test. Le type de navigateur est défini dans la configuration (par exemple Chromium).

7.2 Browser Context

Un browser context représente un environnement de navigation isolé. Le cours mentionne notamment les options liées aux proxies, cookies et permissions.

7.3 Page

Une page représente une page/onglet du navigateur dans un browser context. C'est sur cet objet que l'on effectue des opérations comme goto().

Schéma mental :

Browser → Browser Context → Page

8. Création manuelle d'un context et d'une page

```
import { test } from '@playwright/test';

test("Browser Context Test", async ({ browser }) => {
  // Browser context options can include proxies, cookies, permissions, etc.
  // Open a new browser context.
  // The browser type is defined in playwright.config.js.
  const context = await browser.newContext();

  // Open a new page within the browser context.
  const page = await context.newPage();

  // Navigate to the specified URL.
  await page.goto("https://www.google.com/");
});
```

Ici, le test reçoit la fixture browser, puis crée explicitement le browser context et la page.

9. La fixture page

Le cours montre qu'on peut demander directement la fixture page dans les paramètres de la fonction de test. Playwright gère alors la création du contexte et de la page pour le test.

```
test("Page Fixture Test", async ({ page }) => {
  // Playwright automatically creates the browser context and page.
  await page.goto("https://www.google.com/");
});
```

C'est généralement la forme la plus simple pour commencer un test UI.

10. Navigation avec page.goto()

page.goto(url) demande à la page de naviguer vers l'URL indiquée. Le cours utilise notamment Google et une page de test de connexion comme exemples.

Exemple : await page.goto("https://www.google.com/");

11. Le fichier playwright.config.js

La configuration centralise les paramètres communs aux tests. Le chapitre utilise defineConfig() et une configuration minimale.

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  testDir: './tests',

  // Set the default timeout for all tests to 40 seconds.
  timeout: 40 * 1000,

  // Set the default timeout for assertions.
  expect: {
    timeout: 40 * 1000,
  },

  // Generate an HTML report after test execution.
  reporter: 'html',

  /* Shared settings for all projects. */
  use: {
    // Use Chromium as the browser.
    browserName: 'chromium',

    /* Collect a trace when retrying a failed test. */
    trace: 'on-first-retry',
  },
});
```

12. Explication de la configuration

testDir : Indique le dossier dans lequel Playwright recherche les tests.

timeout : Définit le délai d'attente par défaut d'un test. Ici : 40 secondes.

expect.timeout : Définit le délai d'attente par défaut des assertions.

reporter: 'html' : Demande la génération d'un rapport HTML après l'exécution.

use.browserName : Définit le navigateur utilisé par défaut. Ici : Chromium.

use.trace : Le cours utilise 'on-first-retry' pour collecter une trace lorsqu'un test est relancé après un échec.

13. Exécuter les tests

Commande indiquée dans le cours :

Par défaut, le cours indique que l'exécution est headless : le navigateur s'exécute sans interface visible.

Pour voir l'exécution en temps réel, le cours utilise :

npx playwright test --headed

14. test.only()

test.only() permet d'exécuter uniquement le test sélectionné dans le fichier, tandis que les autres tests de ce fichier sont ignorés.

Exemple : test.only("Page Playwright Test", async ({ page }) => { ... });

À retenir : pratique pour le développement et le débogage, mais il faut éviter de laisser test.only() dans une suite destinée à être exécutée complètement.

15. Assertions avec expect()

Une assertion vérifie qu'un résultat réel correspond à un résultat attendu. Le cours introduit expect() avec une vérification du titre de page.

```
import { expect, test } from '@playwright/test';

test("Page Fixture Test", async ({ page }) => {
  await page.goto("https://www.google.com/");

  console.log(await page.title());

  await expect(page).toHaveTitle("Google");
});
```

La ligne await expect(page).toHaveTitle("Google") vérifie que le titre de la page correspond à la valeur attendue.

16. console.log() et assertions : deux rôles différents

console.log() sert à afficher une information dans la console, par exemple le titre retourné par page.title(). Une assertion comme toHaveTitle() sert à faire échouer le test si le résultat attendu n'est pas obtenu.

17. Plusieurs navigateurs

Le chapitre introduit la configuration de plusieurs navigateurs. Dans l'exemple fourni, browserName est configuré sur chromium. La structure Playwright permet ensuite de définir plusieurs projets de configuration pour exécuter les mêmes tests dans différents navigateurs.

Le principe à retenir : le test décrit le comportement attendu, tandis que la configuration peut déterminer dans quel environnement navigateur ce test est exécuté.

18. Exemple complet du chapitre

```
import { expect, test } from '@playwright/test';

test("Browser Context Test", async ({ browser }) => {
  const context = await browser.newContext();
  const page = await context.newPage();

  await page.goto("https://practicetestautomation.com/practice-test-login/");
  console.log(await page.title());
});

test("Page Fixture Test", async ({ page }) => {
  await page.goto("https://www.google.com/");

  console.log(await page.title());

  await expect(page).toHaveTitle("Google");
});
```

19. Comprendre le cycle d'un test

4. Playwright charge la configuration.
5. Le runner découvre les fichiers de tests dans testDir.
6. Le test demande des fixtures comme browser ou page.
7. Playwright prépare l'environnement nécessaire.
8. Le test exécute les actions avec await.
9. Les assertions vérifient les résultats.
10. Le runner produit le résultat et, selon la configuration, un rapport.

20. Erreurs et points d'attention

Cette section est un complément pédagogique : elle ne figure pas explicitement dans les notes du cours.

- Utiliser une seule convention de modules dans le projet : par exemple import/export avec export default, ou require/module.exports selon la configuration du projet.
- Installer les dépendances dans le bon dossier du projet Playwright.
- Vérifier que @playwright/test est bien installé lorsque les tests importent test et expect depuis ce package.
- Éviter les doublons de versions de Playwright dans différents node_modules.
- Si npx demande d'installer un package alors qu'il est déjà attendu dans le projet, vérifier le dossier courant et package.json.
- Si Playwright indique que test() n'est pas attendu à cet endroit, vérifier notamment les versions multiples de @playwright/test et l'endroit depuis lequel la commande est exécutée.

21. Mémo des concepts

Concept	À retenir
---------	-----------

<code>test()</code>	Déclare un cas de test.
<code>async / await</code>	Permet d'attendre les opérations asynchrones du scénario.
<code>browser</code>	Instance du navigateur utilisée par le test.
<code>browser context</code>	Environnement de navigation isolé.
<code>page</code>	Page/onglet sur lequel les actions sont effectuées.
<code>page.goto()</code>	Navigue vers une URL.
<code>expect()</code>	Déclare une vérification du résultat attendu.
<code>toHaveTitle()</code>	Vérifie le titre de la page.
<code>test.only()</code>	Exécute uniquement le test ciblé.
<code>playwright.config.js</code>	Centralise les paramètres d'exécution.
<code>headless</code>	Exécution sans afficher l'interface du navigateur.
<code>headed</code>	Exécution avec interface visible.

22. Checklist d'apprentissage

- ☐ Je sais initialiser un projet avec `npm init playwright`.
- ☐ Je sais identifier `node_modules`, tests et `package.json`.
- ☐ Je sais créer un test avec `test()`.
- ☐ Je comprends pourquoi `async` et `await` sont utilisés.
- ☐ Je comprends `browser`, `browser context` et `page`.
- ☐ Je sais créer un context et une page manuellement.
- ☐ Je sais utiliser la fixture `page`.
- ☐ Je comprends les principales options du fichier de configuration.
- ☐ Je sais lancer `npx playwright test`.
- ☐ Je sais lancer `npx playwright test --headed`.
- ☐ Je sais isoler un test avec `test.only()`.
- ☐ Je sais écrire une première assertion avec `expect()`.
- ☐ Je comprends la différence entre `console.log()` et une assertion.

23. Exercices conseillés

1. Créer un test qui ouvre Google avec la fixture `page`.
2. Afficher le titre de la page dans la console avec `page.title()`.
3. Ajouter une assertion `toHaveTitle()`.
4. Créer un deuxième test utilisant `browser`, `browser.newContext()` et `context.newPage()`.
5. Lancer les tests en mode `headless` puis en mode `headed`.
6. Utiliser `test.only()` pour exécuter un seul test.

7. Modifier temporairement le titre attendu pour observer un échec d’assertion.
8. Modifier timeout dans la configuration et observer son effet.
9. Explorer le rapport HTML généré après l’exécution.

24. Sources du chapitre

Source principale : Notes.docx fourni avec le cours. Les thèmes couverts dans les notes sont : introduction et fonctionnalités, organisation du cours, installation de Node.js/VS Code, initialisation du projet, structure du projet, premier test, async/await, Browser Context et Page fixtures, configuration, exécution headless/headed, test.only(), assertions et configuration multi-navigateurs.

Documentation officielle mentionnée dans les notes : <https://playwright.dev/docs/actionability>

25. Résumé final

Le modèle mental à retenir à la fin de ce premier chapitre est simple : un projet Playwright possède une configuration, un dossier de tests et des dépendances. Un test est déclaré avec `test()`, s’exécute de façon asynchrone avec `async/await`, utilise des fixtures comme `browser` ou `page`, navigue avec `page.goto()`, puis vérifie le résultat avec `expect()`. La configuration détermine notamment le dossier des tests, les timeouts, le reporter, le navigateur et la trace.